



6049B – VERILOG HDL & PLD **LAB MANUAL**

DIPLOMA -2021 REVISION

Exp No. 1

VERILOG GATE LEVEL DESCRIPTION OF LOGIC GATES

Aim:

To simulate and verify Verilog gate level description of different logic gates (AND, OR, NOT, XOR, NAND, NOR)

Requirements:

Xilinx ISE software

Verilog Code:

AND GATE

```
module and2 (in1, in2, out1);  
input in1, in2;  
output out1;  
and g1(out1, in1, in2);  
endmodule
```

OR GATE

```
module or2 (in1, in2, out1);  
input in1, in2;  
output out1;  
or g1(out1, in1, in2);  
endmodule
```

XOR GATE

```
module xor2 (in1, in2, out1);  
input in1, in2;  
output out1;  
xor g1(out1, in1, in2);  
endmodule
```

NOT GATE

```
module notgate (in1, out1);  
input in1;  
output out1;  
not g1(out1, in1);  
endmodule
```

NAND GATE

```
module nand2 (in1, in2, out1);  
input in1, in2;  
output out1;  
nand g1(out1, in1, in2);  
endmodule
```

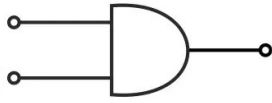
NOR GATE

```
module nor2 (in1, in2, out1);  
input in1, in2;  
output out1;  
nor g1(out1, in1, in2);  
endmodule
```

Result:

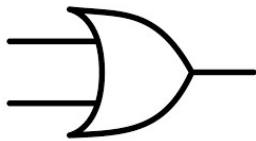
Simulated and verified Verilog gate level description of different logic gates

AND GATE



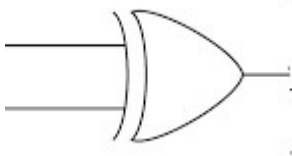
in1	in2	out1
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE



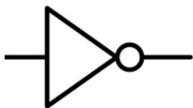
in1	in2	out1
0	0	0
0	1	1
1	0	1
1	1	1

XOR GATE



in1	in2	out1
0	0	0
0	1	1
1	0	1
1	1	0

NOT GATE



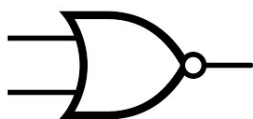
in1	out1
0	1
1	0

NAND GATE



in1	in2	out1
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE



in1	in2	out1
0	0	1
0	1	0
1	0	0
1	1	0

Exp No. 2

VERILOG GATE LEVEL DESCRIPTION OF BOOLEAN EXPRESSIONS

Aim:

To simulate and verify Verilog gate level description of the following Boolean expressions

1. $Y = AB + BC$
2. $Y = A'B + AB'$

Requirements:

Xilinx ISE software

Verilog Code:

Expression 1

```
module expr1 (A, B, C, Y)
input A, B, C
output Y;
wire w1, w2;

and g1(w1, A, B);
and g2(w2, B, C);
or g3 (Y, w1, w2);
endmodule
```

Expression 2

```
module expr2 (A, B, Y);
input A, B;
output Y;
wire An, Bn, w1, w2;

not g1 (An, A);
not g2 (Bn, B);
```

```
and g3 (w1, An, B);  
and g4 (w2, A, Bn);  
or g5 (Y, w1, w2);  
endmodule
```

Result:

Simulated and verified Verilog gate level description of Boolean expressions

CIRCUIT DIAGRAM & TRUTH TABLE

Exp No. 3

VERILOG GATE LEVEL DESCRIPTION OF ONE BIT FULL ADDER

Aim:

To simulate and verify Verilog gate level description of one bit full adder

Requirements:

Xilinx ISE software

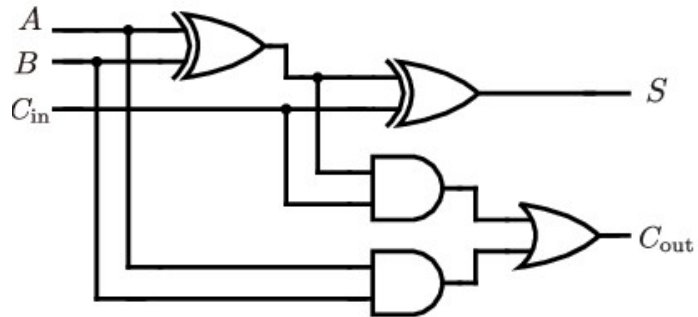
Verilog Code:

```
module one_bit_FA (A, B, Cin, S, Cout);  
input A, B, Cin;  
output S, Cout;  
wire w1, w2, w3  
  
xor g1 (w1, A, B);  
xor g2 (S, Cin, w1);  
and g3 (w2, w1, Cin);  
and g4 (w3, A, B);  
or g5 (Cout, w2, w3);  
endmodule;
```

Result:

Simulated and verified Verilog gate level description of one bit full adder.

ONE BIT FULL ADDER



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Exp No. 4

4:1 MULTIPLEXR

Aim:

To simulate and verify Verilog gate level description of 4:1 multiplexer

Requirements:

Xilinx ISE software

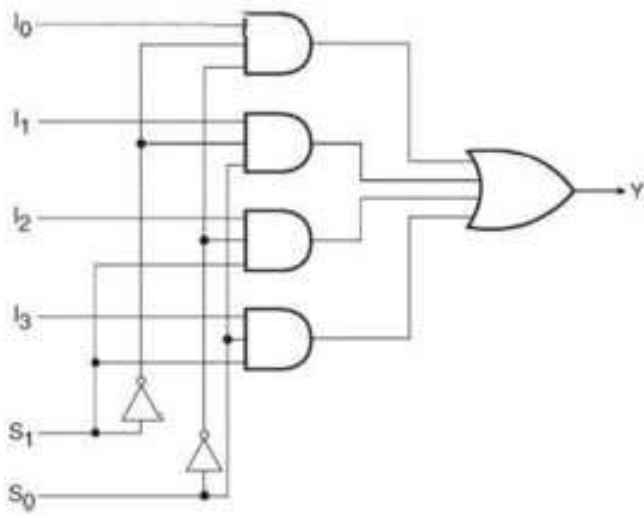
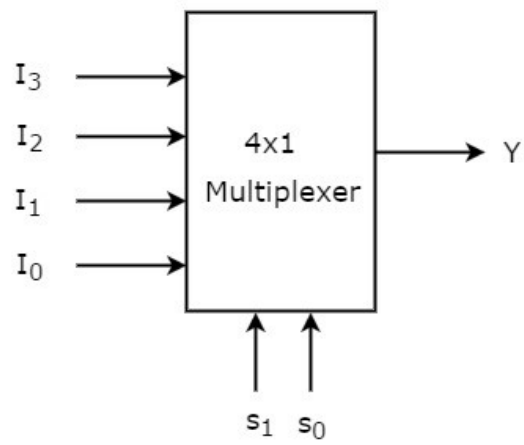
Verilog Code:

```
module 4x1_mux (i0, i1, i2, i3, s0, s1, y);  
input i0, i1, i2, i3, s0, s1;  
output y;  
wire s0_n, s1_n, y0, y1, y2, y3;  
  
not g1 (s1_n, s1);  
not g2 (s0_n, s0);  
and g3 (y0, i0, s1_n, s0_n);  
and g4 (y1, i1, s1_n, s0);  
and g5 (y2, i2, s1, s0_n);  
and g6 (y3, i3, s1, s0);  
or g7 (y, y0, y1, y2, y3);  
endmodule
```

Result:

Simulated and verified Verilog gate level description of 4:1 multiplexer.

4:1 MULTIPLEXER



s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Exp No. 5

2 – 4 DECODER

Aim:

To simulate and verify Verilog gate level description of 2 – 4 decoder.

Requirements:

Xilinx ISE software

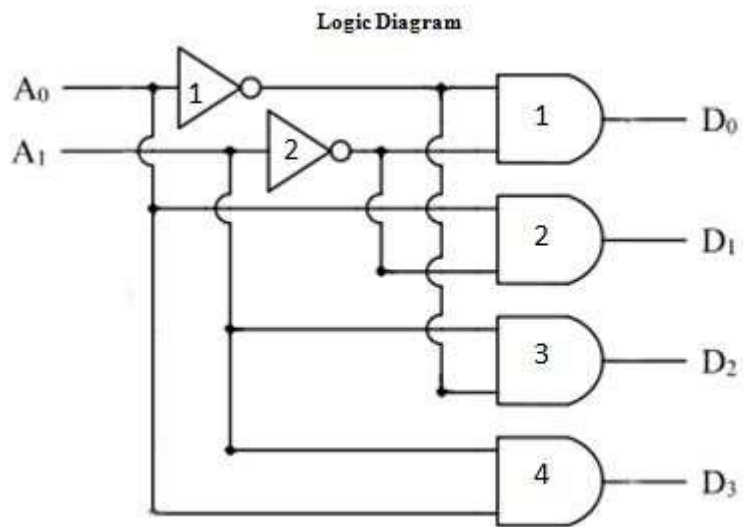
Verilog Code:

```
module 2_4decoder (A0, A1, D0, D1, D2, D3);  
input A0, A1;  
output D0, D1, D2, D3;  
wire A0_n, A1_n;  
  
not g1 (A0_n, A0);  
not g2 (A1_n, A1);  
and g3 (D0, A1_n, A0_n);  
and g4 (D1, A1_n, A0);  
and g5 (D2, A1, A0_n);  
and g6 (D3, A1, A0);  
endmodule
```

Result:

Simulated and verified Verilog gate level description of 2 – 4 decoder.

2 – 4 DECODER



Truth Table

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Equations

$$D_0 = \overline{A_1} \cdot \overline{A_0}$$

$$D_1 = \overline{A_1} \cdot A_0$$

$$D_2 = A_1 \cdot \overline{A_0}$$

$$D_3 = A_1 \cdot A_0$$

Exp No. 6

HALF ADDER USING DATA FLOW MODELLING

Aim:

To simulate and verify Verilog description of half adder using data flow modelling.

Requirements:

Xilinx ISE software

Verilog Code:

```
module HA (A, B, S, C);
```

```
input A, B;
```

```
output S, C;
```

```
assign S = A ^ B ;
```

```
assign C = A & B;
```

```
endmodule
```

Test Bench:

```
module HA_tb ;
```

```
reg A, B;
```

```
wire S, C;
```

```
HA u1 (A, B, S, C);
```

```
initial
```

```
begin
```

```
#5 A=0; B=0;
```

```
#5 A=0; B=1;
```

```
#5 A=1; B=0;
```

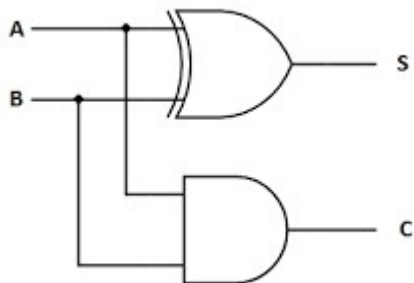
```
#5 A=1; B=1;
```

```
#5 $finish;  
end  
endmodule
```

Result:

Simulated and verified the Verilog description for half adder using data flow modelling.

HALF ADDER



TRUTH TABLE

INPUT		OUTPUT	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Exp No. 7

ONE BIT FULL ADDER USING DATA FLOW MODELLING

Aim:

To simulate and verify Verilog description of one bit full adder using data flow modelling.

Requirements:

Xilinx ISE software

Verilog Code:

```
module one_bit_FA (A, B, Cin, S, Cout);  
input A, B, Cin;  
output S, Cout;  
  
assign S = A ^ B ^ Cin;  
assign Cout = ((A ^ B) & Cin) | (A & B);  
endmodule
```

Test Bench:

```
module one_bit_FA_tb ;  
reg A, B, Cin;  
wire S, Cout;  
one_bit_FA u1 (A, B, Cin, S, Cout);  
  
initial  
begin  
#5 A=0; B=0; Cin=0;  
#5 A=0; B=0; Cin=1;  
#5 A=0; B=1; Cin=0;
```



```
#5 A=0; B=1; Cin=1;
#5 A=1; B=0; Cin=0;
#5 A=1; B=0; Cin=1;
#5 A=1; B=1; Cin=0;
#5 A=1; B=1; Cin=1;
#5 $finish;

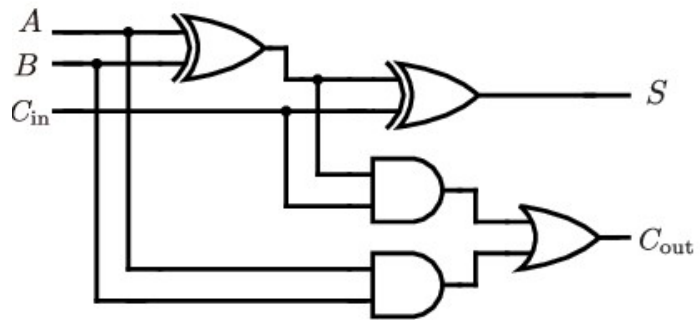
end

endmodule
```

Result:

Simulated and verified the Verilog description for one bit full adder using data flow modelling.

ONE BIT FULL ADDER



Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Exp No. 8

4:1 MUX USING CONDITIONAL OPERATOR

Aim:

To simulate and verify Verilog description of 4:1 MUX using conditional operator.

Requirements:

Xilinx ISE software

Verilog Code:

```
module four_to_one_mux (i0, i1, i2, i3, s0, s1, y);  
input i0, i1, i2, i3, s0, s1;  
output y;  
assign y = s1? (s0? i3 : i2) : (s0? i1 : i0);  
endmodule
```

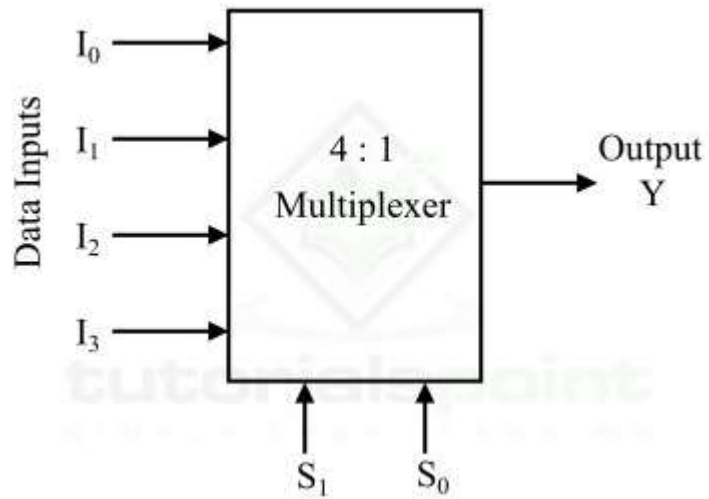
Test Bench

```
module four_to_one_mux_tb;  
reg i0, i1, i2, i3, s0, s1;  
wire y;  
four_to_one_mux u1 (i0, i1, i2, i3, s0, s1, y);  
initial  
begin  
i0=1; i1=0; i2=1; i3=0; s0=0; s1=0;  
#5 s1=0; s0=1;  
#5 s1=1; s0=0;  
#5 s1=1; s0=1;  
#5 $finish;  
end  
endmodule
```

Result:

Simulated and verified Verilog description of 4:1 multiplexer using conditional operator.

4 X 1 MUX



TRUTH TABLE

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Exp No. 9

D FLIP FLOP USING DATA FLOW MODELLING

Aim:

To simulate and verify Verilog description of D Flip Flop using data flow modelling.

Requirements:

Xilinx ISE software

Verilog Code:

```
module DFF (D, clk, Q, Qb);  
input D, clk;  
output Q, Qb;  
  
assign Q = clk? (D? 1 : 0) : (~Qb);  
assign Qb = ~Q;  
endmodule
```

Test Bench:

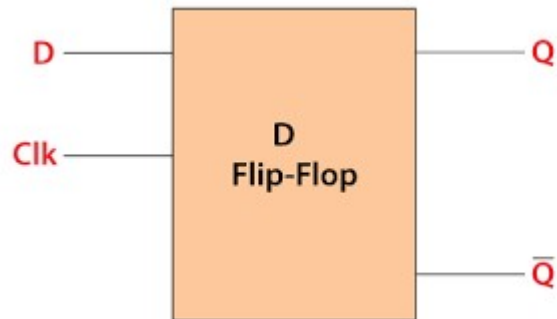
```
module DFF_tb;  
reg D, clk;  
wire Q, Qb;  
  
DFF u1 (D, clk, Q, Qb);  
  
initial  
begin  
clk = 1'b0;  
end  
  
always #5 clk = ~clk;
```

```
initial
begin
#5 D = 0;
#5 D = 1;
#5 D = 1;
#5 D = 0;
#5 $finish;
end
endmodule
```

Result:

Simulated and verified Verilog description of D Flip Flop using data flow modelling.

D FLIP FLOP



Truth Table

Inputs		Outputs	
Clk	D	Q	Qb
0	X	No change	
1	0	0	1
1	1	1	0

Exp No. 10

HALF SUBTRACTOR AND FULL SUBTRACTOR USING BEHAVIOURAL MODELLING

Aim:

To simulate and verify Verilog description of half subtractor and full subtractor using behavioral modelling.

Requirements:

Xilinx ISE

Verilog Code for Half Subtractor:

```
module half_sub (X, Y, D, B);  
    input X, Y;  
    output reg D, B;  
    always @ (*)  
        begin  
            D = X ^ Y;  
            B = ~X & Y;  
        end  
endmodule
```

Verilog Code for Full Subtractor:

Code 1:

```
module full_sub (X, Y, Bin, D, Bout);  
    input X, Y, Bin;  
    output reg D, Bout;  
  
    half_sub u1 (X, Y, w1, w2);
```



```

        half_sub u2 (w1, Bin, D, w3);

        assign Bout = w2 | w3;

    endmodule

```

Code 2;

```

module full_sub (X, Y, Bin, D, Bout);

    input X, Y, Bin;

    output reg D, Bout;

    always @ (*)

    begin

        D = X ^ Y ^ Bin;

        Bout = (~X&Y) | (~X & Bin) | (Y &~Bin);

    end

endmodule

```

Test Bench for Full Subtractor:

```

module full_sub_tb;

    reg X, Y, Bin;

    wire D, Bout;

    full_sub u1 (X, Y, Bin, D, Bout);

    initial

        begin

            X = 0; Y = 0; Bin = 0;

            #5 X = 0; Y = 0; Bin = 1;

            #5 X = 0; Y = 1; Bin = 0;

            #5 X = 0; Y = 1; Bin = 1;

        end

endmodule

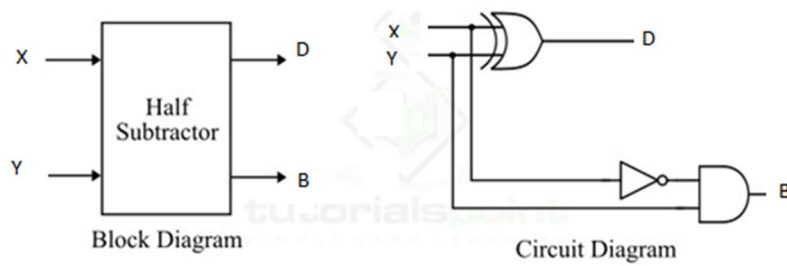
```

```
#5 X = 1; Y = 0; Bin = 0;  
#5 X = 1; Y = 0; Bin = 1;  
#5 X = 1; Y = 1; Bin = 0;  
#5 X = 1; Y = 1; Bin = 1;  
end  
endmodule
```

Result:

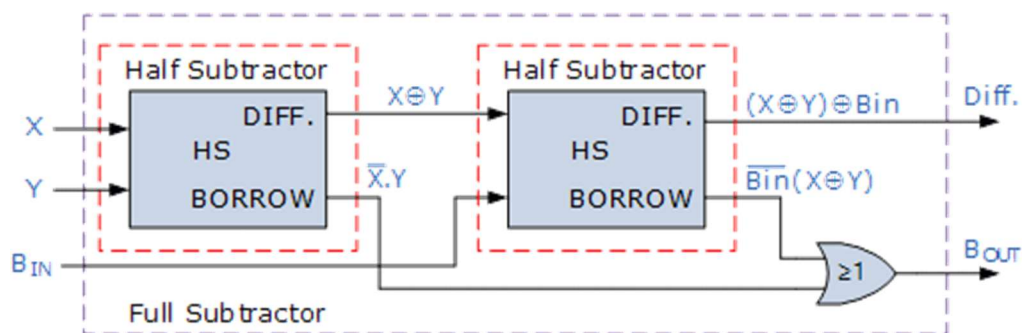
Simulated and verified Verilog description of half subtractor and full subtractor.

HALF SUBTRACTOR



X	Y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

FULL SUBTRACTOR



X	Y	B _{IN}	D	B _{OUT}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Exp No. 11

4 X 1 MUX USING BEHAVIOURAL MODELLING

Aim:

To simulate and verify Verilog description for 4 X 1 mux using behavioral modelling

Requirements:

Xilinx ISE

Verilog Code:

```
module m41 ( i0, i1, i2, i3, s0, s1, y);  
input i0, i1, i2, i3, s0, s1;  
output reg y;  
  
always @ (*)  
begin  
    case ({s1, s0})  
        2'b00 : out <= i0;  
        2'b01 : out <= i1;  
        2'b10 : out <= i2;  
        2'b11 : out <= i3;  
    endcase  
end  
endmodule
```

Test Bench

```
module m41_tb;
reg i0, i1, i2, i3, s0, s1;

wire y;

m41 u1 (i0, i1, i2, i3, s0, s1, y);

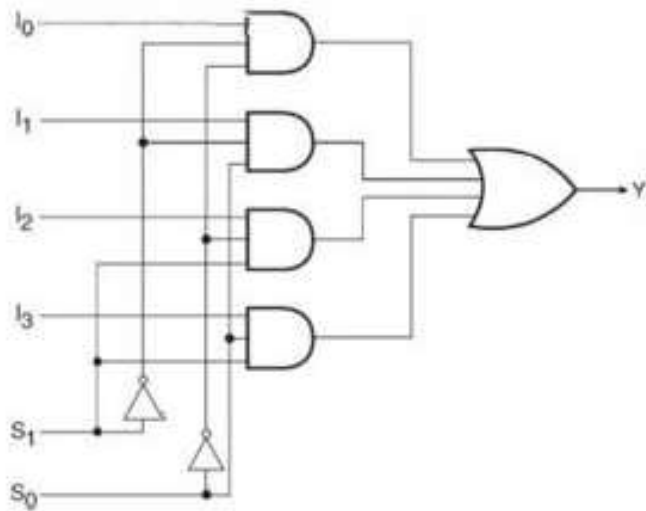
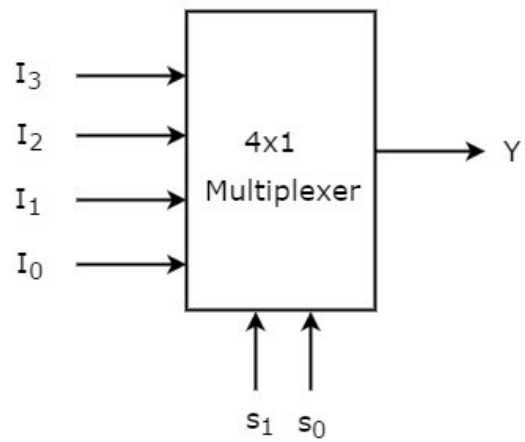
initial
begin
i0=1; i1=0; i2=1; i3=0; s0=0; s1=0;
#5 s1=0; s0=1;
#5 s1=1; s0=0;
#5 s1=1; s0=1;
$finish;
end

endmodule
```

Result:

Simulated and verified Verilog description of 4:1 multiplexer using behavioural modelling.

4:1 MULTIPLEXER



s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Exp No. 12

J K FLIP FLOP

Aim:

To simulate and verify Verilog description for J K Flip Flop using behavioral modelling

Requirements:

Xilinx ISE

Verilog Code:

```
module jkff (clk, rst, j, k, q, q_bar);
input clk, rst, j, k;
output reg q, q_bar;

always @(posedge clk)
begin
    if (!rst)
        begin
            q <= 1'b0; // Reset state
        end
    else
        begin
            case ({j, k})
                2'b00: q <= q; // No change (Hold state)
                2'b01: q <= 1'b0; // Reset (Q=0)
                2'b10: q <= 1'b1; // Set (Q=1)
                2'b11: q <= ~q; // Toggle (Q changes state)
                default: q <= q; // Default case
            endcase
        end
    end
end
```

```
        end
    end
    assign q_bar = ~q;
endmodule
```

Test bench

```
module tb_jkff( );
    reg clk, rst, j, k;
    wire q, q_bar;
    jkff u1 (clk, rst, j, k, q, q_bar);
```

```
    initial
        begin
            clk = 1'b0;
            j = 1'b0;
            k = 1'b0;
            rst = 1'b0;
        end
    always #5 clk = ~clk;
```

```
    initial
        begin
            #10 rst = 1'b1;
            #10 j = 1'b0; k = 1'b0;
            #10 j = 1'b0; k = 1'b1;
            #10 j = 1'b1; k = 1'b0;
            #10 j = 1'b1; k = 1'b1;
            #10 j = 1'b1; k = 1'b1;
            #10 j = 1'b0; k = 1'b0;
```

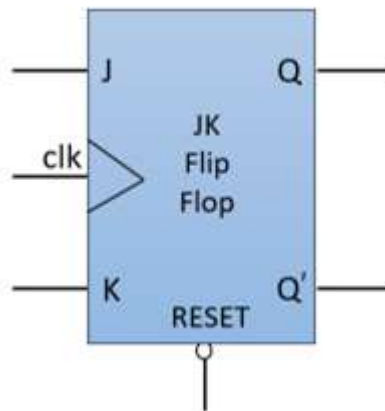


```
#20 $finish;  
end  
endmodule
```

Result:

Simulated and verified Verilog description for JK Flip Flop using behavioral modelling

JK FLIP FLOP



TRUTH TABLE

Clk	J	K	Q	Q'	State
1	0	0	Q	Q'	No change in state
1	0	1	0	1	Resets Q to 0
1	1	0	1	0	Sets Q to 1
1	1	1	-	-	Toggles

Exp No. 13

4 BIT SHIFT REGISTER

Aim:

To simulate and verify Verilog description for 4 bit shift register using behavioral modelling

Requirements:

Xilinx ISE

Verilog Code:

```
module shiftreg_4bit(in, clk, clr, out);
input clk, clr, in;
output reg out;
reg B, C, D;
always (posedge clk or negedge clr)
begin
    if(!clr)
        begin
            B<=0; C<=0; D<=0; out<=0;
        end
    else
        begin
            out <= D;
            D <= C;
            C <= B;
            B <= in;
        end
    end
end
endmodule
```

Test bench

```
module shiftreg_4bit_tb;

reg in, clk,clr;

wire out;


shiftreg_4bit u1(in, clk, clr, out);


initial
    begin
        clk = 1'b0;
        #2 clr = 0;
        #5 clr = 1;
    end

always #5 clk = ~clk;

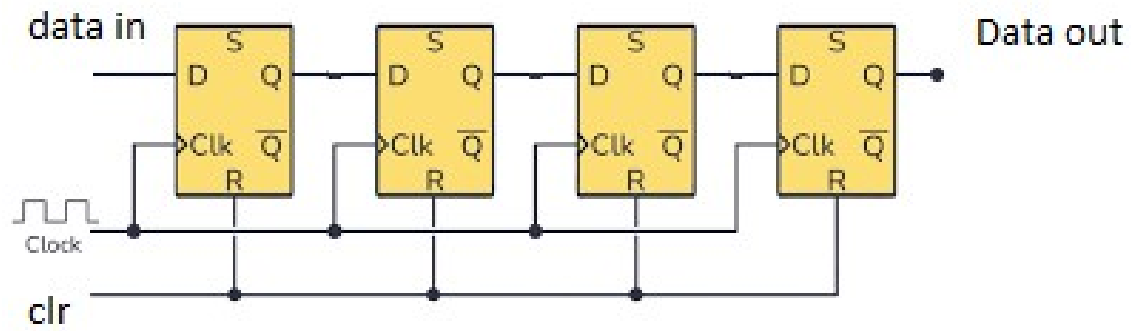

initial
begin
    #10 in=1;
    #10 in=0;
    #10 in=1;
    #10 in=0;
    #30 clr = 1'b0;
    #30 $finish;
end

endmodule
```

Result:

Simulated and verified Verilog description for 4 bit shift register using behavioral modelling

4 BIT SHIFT REGISTER



Exp No. 14

4 BIT RIPPLE COUNTER

Aim:

To simulate and verify Verilog description for 4 bit ripple counter using behavioral modelling

Requirements:

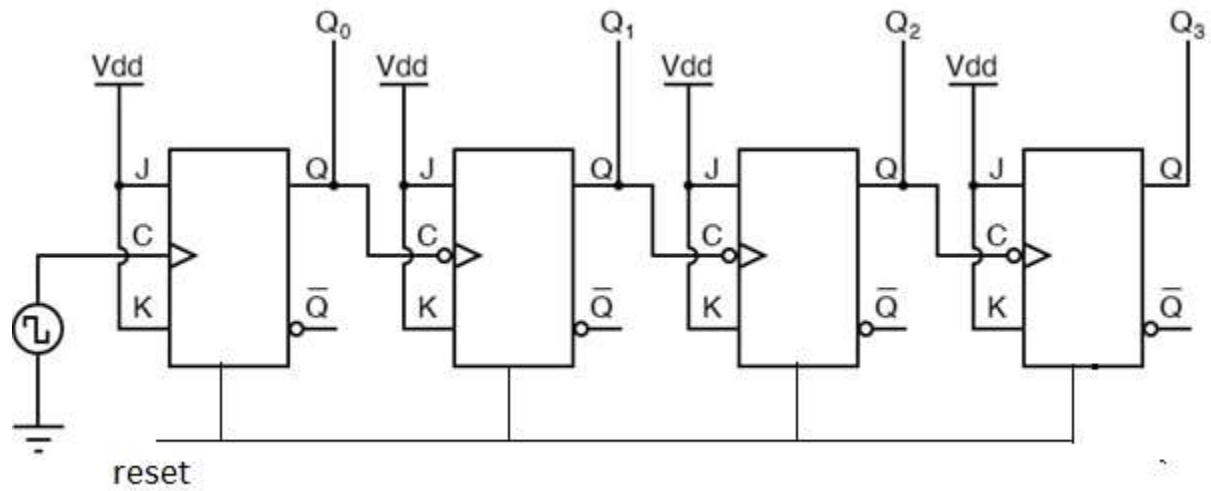
Vivado software, Boolean Board

Verilog Code:

```
module ripple_counter_4bit_behavioral (clk, reset, count);
    input clk, reset;
    output reg [3:0] count;

    always @(posedge clk or posedge reset)
    begin
        if (reset)
            begin
                count <= 4'b0000;
            end
        else
            begin
                count <= count + 1;
            end
        end
    end
endmodule
```

4 BIT RIPPLE COUNTER



TRUTH TABLE

Clk	Q3	Q2	Q1	Q0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

Exp No. 15

IMPLEMENTATION OF BINARY TO GRAY CODE CONVERTER USING TRAINER KIT

Aim:

To implement and verify Verilog description for binary to gray code converter using Boolean Board

Requirements:

Vivado software, Boolean Board

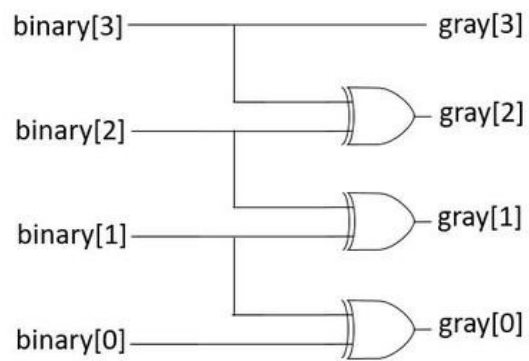
Verilog Code:

```
module bin_to_gray(b,g);  
input [3:0] b;  
output [3:0] g;  
    assign g[3] = b [3];  
    assign g[2] = b [3] ^ b[2];  
    assign g[1] = b [2] ^ b[1];  
    assign g[0] = b [1] ^ b[0];  
endmodule
```

Result:

Implemented a binary to gray code converter using Boolean Board FPGA Trainer kit.

BINARY TO GRAY CODE CONVERTER



Binary Code	Gray Code
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101
1010	1111
1011	1110
1100	1010
1101	1011
1110	1001
1111	1000

Exp No. 16

BCD TO SEVEN SEGMENT DECODER USING TRAINER KIT

Aim:

To implement and verify Verilog description for BCD to seven segment decoder using Boolean Board

Requirements:

Vivado software, Boolean Board

Verilog Code:

```
module bcd_7segment(B, ctrl, seg7);
    input [3:0] B;
    output [3:0] ctrl;
    output reg [6:0] seg7;

    assign ctrl = 4'b1110;
    always @(B)
        case(B)
            0: seg7 = 7'b0000001;
            1: seg7 = 7'b1001111;
            2: seg7 = 7'b0010010;
            3: seg7 = 7'b0000110;
            4: seg7 = 7'b1001100;
            5: seg7 = 7'b0100100;
            6: seg7 = 7'b0100000;
            7: seg7 = 7'b0001111;
            8: seg7 = 7'b0000000;
            9: seg7 = 7'b0000100;
            10: seg7 = 7'b0001000;
            11: seg7 = 7'b1100000;
            12: seg7 = 7'b0110001;
            13: seg7 = 7'b1000010;
            14: seg7 = 7'b0110000;
            15: seg7 = 7'b0111000;
            default : seg7 = 7'b1111111;
        endcase
    endmodule
```

Result: Implemented a BCD to 7 segment decoder using Boolean Board Trainer kit.